

Yes — trimming lines of code/instructions in your agent skills (especially in SKILL.md files and related resources) is one of the most effective ways to make AI agents dramatically more efficient in 2026. The core idea is token efficiency: every unnecessary line costs tokens → slower responses, higher costs, worse attention (lost-in-the-middle problem), and less room for conversation history or tool outputs. The Agent Skills ecosystem (Anthropic's standard, now widely adopted) has built-in architecture and community best practices specifically designed to minimize this. Main Techniques to Trim & Make Your Agent More Efficient

1. Use Progressive Disclosure / Lazy Loading (The #1 Win)

This is the official killer feature of Agent Skills.

- At startup → only lightweight YAML metadata (name + 1-2 sentence description) loads (~50–100 tokens per skill).
- Agent scans descriptions to decide relevance → only then loads the full SKILL.md (~2–5k tokens max recommended).
- Only when really needed → Claude reads linked reference files (e.g. REFERENCE.md, EXAMPLES.md, schemas) or executes scripts (code never loads into context — just output!).

→ Result: You can have 50+ skills installed with almost zero idle token cost (90%+ reduction reported in many repos).

→ Trim tip: Keep SKILL.md body under 200–500 lines (Anthropic/community consensus). If it grows bigger → split into separate referenced files.

2. Keep SKILL.md as a "Map", Not a Wall of Text

Community pattern from production users (e.g. Reddit threads, wshobson/agents, muratcankoylan repo):

- Treat SKILL.md like an index/table of contents.
- Put only: activation triggers, core rules, brief how-to, and links to details.
- Move examples, long lists, schemas, full workflows → separate files that load on-demand.
- Goal: Cold-start activation should load <500 lines total (test this by clearing context and measuring).
→ Many users report 4–5× token savings just by refactoring bloated skills this way.

3. Offload Heavy Stuff to Executable Scripts

Scripts in the skill folder can run via bash/tool → code never enters context, only output does.

→ Perfect for sorting, validation, parsing, compression — way cheaper than making Claude re-generate equivalent logic every time.

4. Other Trimming & Compression Tricks

- Context compression skill — use dedicated skills (e.g. from [muratcankoylan/Agent-Skills-for-Context-Engineering](#)) that teach your agent how to summarize/compact previous outputs, mask noise, or use KV-cache tricks.
- Attention budgeting — curate high-signal tokens only; remove fluff words, redundant explanations.
- Filesystem offloading — store plans/tool results in files instead of context.
- Single well-crafted example > paragraphs — one strong example often replaces 200–300 lines of description.

Quick Before/After Example Bloated (bad, ~1200+ tokens when active):

markdown

Long Skill

When user says X do this 30-step process...

Step 1: ...

Step 2: ... (very verbose)

... (tons of text)

Also remember these 50 edge cases: ...

Trimmed & Efficient (recommended):

markdown

Efficient Skill

****Activates on:**** "optimize context", "compress", "trim tokens", "fix attention"

****Core Rule:**** Always apply progressive disclosure. Load only what's needed.

****Steps Overview:****

1. Evaluate relevance → 2. Load full module if match → 3. Execute compression script

See: COMPRESSION_GUIDE.md (detailed techniques)

EXAMPLES.md (before/after cases)

Run: compress.py for automatic summarization

→ Massive token & speed win. Bottom line: Yes, aggressively trim — but do it smartly with progressive disclosure + file splitting + scripts.

This turns agents from "kinda works, burns tokens" → "blazing fast, reliable at scale". If you share a specific skill/folder you're working on (or paste a bloated SKILL.md snippet), I can help refactor it for max efficiency!